

Projet : SecNotes DevSecOps

3 Pipelines CI/CD · SAST · Docker · Kubernetes · Terraform

Étudiante : **Hasna Daoui**

Date : 21 May 2026

Plateforme : GitLab CI/CD · Runner Local

Pipeline 1

Todo App — CI/CD Complet

Build · Test · Security · Package · Deploy

■ PASSED

Pipeline 2

MongoDB CRUD — Docker + K8s

Build · Push · Deploy-Kubernetes · EC2 · Terraform

■ PASSED

Pipeline 3

SecNotes — SAST SonarQube

Install · Test · SAST · Build · ZAP · Terraform

■ PASSED

Pipelines réalisés	3	Durée totale (cumulée)	~2h15min
Outils de sécurité	5	Vulnérabilités détectées	7 (SAST)
Images Docker buildées	3	Déploiements Kubernetes	2
Scripts d'automatisation	2	Environnements couverts	Staging · K8s · EC2

Table des Matières

1. Introduction & Architecture DevSecOps Globale
2. Pipeline 1 — Todo App : CI/CD Complet
 - 2.1 Description & Stages
 - 2.2 Docker & Containerisation
 - 2.3 Résultats & Screenshots
3. Pipeline 2 — MongoDB CRUD : Docker + Kubernetes + Terraform
 - 3.1 Description & Stages
 - 3.2 Docker & Docker Compose
 - 3.3 Déploiement Kubernetes
 - 3.4 Infrastructure Terraform (EC2)
 - 3.5 Screenshots
4. Pipeline 3 — SecNotes : Analyse SAST SonarQube
 - 4.1 Description & Stages
 - 4.2 Docker & Configuration
 - 4.3 Résultats SAST — Vulnérabilités Détectées
 - 4.4 Screenshots
5. Vue Globale : Docker Across All Pipelines
6. Scripts d'Automatisation
7. Conclusion Générale

1. Introduction & Architecture DevSecOps Globale

Ce rapport présente l'ensemble des travaux pratiques réalisés dans le cadre du module **DevSecOps**. Trois pipelines CI/CD complets ont été mis en œuvre sur la plateforme **GitLab CI/CD**, couvrant progressivement les piliers fondamentaux du DevSecOps : intégration continue, livraison continue, containerisation Docker, orchestration Kubernetes, analyse statique de sécurité (SAST) et infrastructure-as-code (Terraform).

L'approche **Shift-Left Security** est appliquée tout au long des pipelines : les contrôles de sécurité sont intégrés le plus tôt possible dans le cycle de développement, avant même le déploiement en production.

Architecture Globale :

Composant	Rôle	Pipelines concernés
GitLab CI/CD	Orchestration des pipelines, gestion des runners	P1, P2, P3
Docker	Containerisation des applications	P1, P2, P3
GitLab Registry	Stockage des images Docker	P1, P2
Kubernetes (Docker Desktop)	Orchestration des conteneurs	P2, P3
SonarQube	Analyse SAST statique du code	P3
Terraform	Infrastructure-as-Code (AWS EC2)	P2, P3
OWASP ZAP	Analyse DAST dynamique	P3
Gitleaks	Détection de secrets dans le code	P1
npm audit	Scan des vulnérabilités des dépendances	P1, P3
AWS EC2	Déploiement cloud (instance distante)	P2

2. Pipeline 1 — Todo App : CI/CD Complet

Le **Pipeline 1** est le premier pipeline CI/CD réalisé, portant sur une application **Todo App Node.js**. Il implémente un pipeline complet en 5 stages couvrant la construction, les tests, l'analyse de sécurité, le packaging Docker et le déploiement en staging.

2.1 Description & Stages

Stage	Job	Outil	Description
build	build-job	Node.js 18	npm install + vérification syntaxe JS (node -c)
test	test-job	Jest	Tests unitaires (jest --runInBand)
security	security-secrets-scan	Gitleaks	Détection de secrets exposés dans le code source
security	security-dependencies-scan	npm audit	Scan des CVE dans les dépendances (level=high)
package	package-docker	Docker DinD	Build + push de l'image vers GitLab Container Registry
deploy	deploy-staging	Docker	Pull + run de l'image en environnement staging (port 8001)

2.2 Docker & Containerisation

L'application Todo App est containerisée avec un **Dockerfile** optimisé basé sur **node:18-alpine** (image légère). L'image est construite et publiée sur le **GitLab Container Registry** avec deux tags : la version sémantique (1.0.0) et latest.

Dockerfile Pipeline 1 :

```
FROM node:18-alpine WORKDIR /app COPY package*.json ./ RUN npm install --production COPY . . EXPOSE 8000 CMD ["node","server.js"]
```

Propriété	Valeur
Image de base	node:18-alpine (image minimaliste)
Port exposé	8000
Commande	node server.js
Registry	registry.gitlab.com/cyber2026/todo-app
Tags	1.0.0 et latest
Déploiement	docker run -d --name todo-staging -p 8001:8000

2.3 Résultats & Screenshot

Le pipeline Pipeline 1 s'est exécuté avec succès. Les 5 stages ont passé, incluant le scan Gitleaks (aucun secret détecté) et le push Docker vers la registry.

✓ Passed

🕒 00:04:09

📅 3 months ago

#2321425471

Edit .gitlab-ci.yml

main

65bdf241

🔗

branch

🌀

✓✓✓✓✓

📄 ⌵

Figure 1 — Pipeline 1 : Statut PASSED sur GitLab CI/CD

3. Pipeline 2 — MongoDB CRUD : Docker + Kubernetes + Terraform

Le **Pipeline 2** porte sur une application **MongoDB CRUD** (Node.js + MongoDB). Il implémente un pipeline en 3 stages automatisés : construction de l'image Docker, push vers la registry, et déploiement sur **Kubernetes** (Docker Desktop). Un déploiement AWS EC2 via **Terraform** a également été réalisé manuellement.

3.1 Description & Stages

Stage	Job	Outil	Description
build	build:docker	Docker DinD	Build de l'image mongo-crud-app:SHA + latest, push registry
push	push:registry	Docker DinD	Re-tag et push de l'image avec le nom de branche (main)
deploy-kubernetes	deploy:kubernetes	kubectl (Windows)	Apply des manifests K8s, rollout status, vérification pods

3.2 Docker & Docker Compose

L'application est disponible en deux modes de déploiement : via le pipeline CI/CD (Docker + Kubernetes) ou via **Docker Compose** pour le développement local.

Docker Compose (développement local) :

```
version: '3.8' services: mongodb: image: mongo:latest ports: ["27017:27017"] volumes: [mongodb_data:/data/db] app: build: . ports: ["3000:3000"] depends_on: [mongodb] environment: [NODE_ENV=production]
```

Propriété	Valeur
Image de base	node:18-alpine
Port exposé	3000
Base de données	MongoDB (image mongo:latest, port 27017)
Registry	\$CI_REGISTRY/\$CI_PROJECT_PATH/mongo-crud-app
Tags	\$CI_COMMIT_SHA, latest, \$CI_COMMIT_REF_SLUG
Docker Compose	Services: app + mongodb avec volume persistant

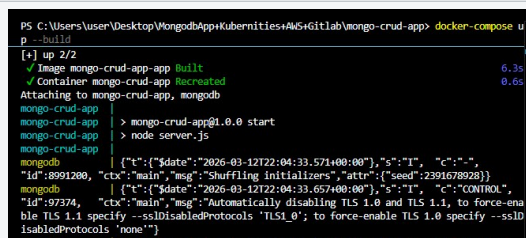


Figure 2 — Test Docker en local (Pipeline 2)

3.3 Déploiement Kubernetes

Le déploiement Kubernetes est réalisé sur **Docker Desktop** avec un namespace dédié *mongodb-app*. Le runner GitLab (shell Windows) exécute les commandes kubectl pour appliquer les manifests et vérifier l'état des pods.

Ressource K8s	Namespace	Description
Namespace	mongodb-app	Isolation de l'application dans son propre namespace
Deployment (app)	mongodb-app	mongo-crud-app — 1 réplique, image depuis GitLab Registry
Deployment (mongo)	mongodb-app	MongoDB — 1 réplique, image mongo:latest
Service (app)	mongodb-app	NodePort exposant l'app sur le nœud K8s
Service (mongo)	mongodb-app	ClusterIP pour accès MongoDB interne
Secret (registry)	mongodb-app	Credentials GitLab Registry pour pull de l'image privée

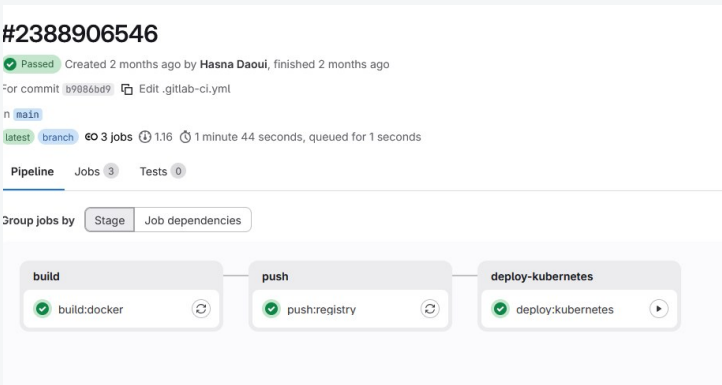


Figure 3 — Pipeline 2 : Statut PASSED (3 stages)

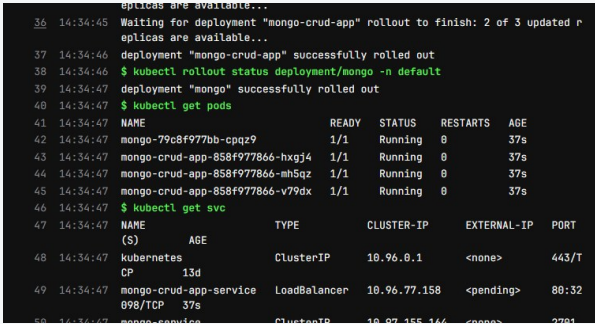


Figure 4 — Pods Kubernetes en état Running (namespace mongodb-app)

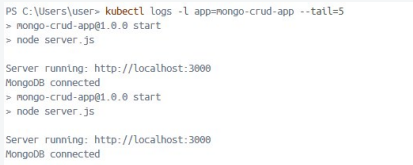


Figure 5 — Connexion MongoDB confirmée dans les logs

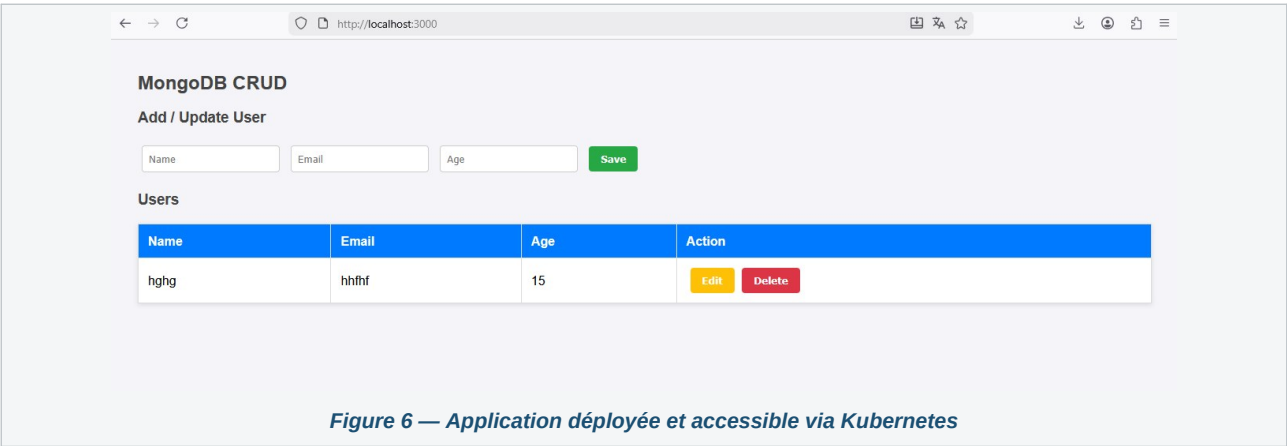


Figure 6 — Application déployée et accessible via Kubernetes

3.4 Infrastructure Terraform (AWS EC2)

En complément du pipeline K8s, un déploiement AWS EC2 a été réalisé via **Terraform**. Une instance EC2 a été provisionnée, l'image Docker y a été déployée avec un réseau *appnet* dédié, et l'accès à l'application a été établi via SSH local port forwarding.

Étape	Action	Résultat
1. EC2	Création instance AWS EC2 via Terraform	Instance opérationnelle (screenshot)
2. SSH	Connexion SSH à l'instance EC2	Accès distant établi
3. Net work	Création du réseau Docker appnet	Isolation réseau conteneurs
4. Mon goDB	docker run mongo sur EC2	MongoDB opérationnel
5. App	docker run mongo-crud-app sur EC2	App accessible sur EC2
6. Port Forwa rd	SSH local port forwarding -L 3000:localhost:3000	App accessible sur navigateur local

3.5 Screenshots Déploiement EC2 & Docker

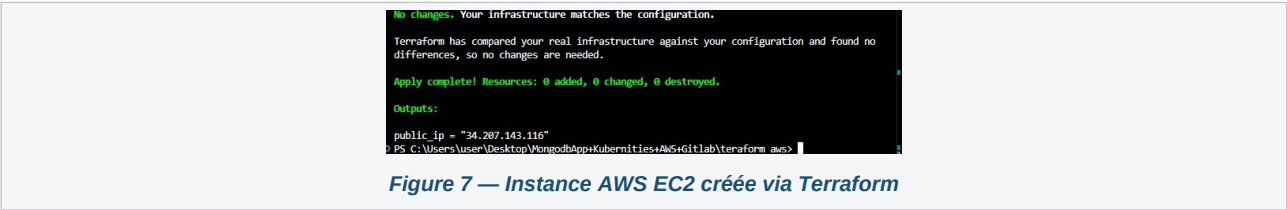


Figure 7 — Instance AWS EC2 créée via Terraform

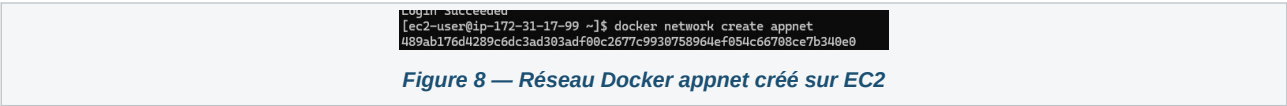


Figure 8 — Réseau Docker appnet créé sur EC2


```
[ec2-user@ip-172-31-17-99 ~]$ docker run -d \
> --name mongo \
> --network appnet \
> -p 27017:27017 \
> mongo:7
Unable to find image 'mongo:7' locally
7: Pulling from library/mongo
b1c8a2e8d2ea: Pull complete
7e6daea0607a: Pull complete
488b7536aa90: Pull complete
053e1e2811d6: Pull complete
f80d1f18be1a: Pull complete
70624009c824: Pull complete
c1575978db83: Pull complete
e7873cabff83: Pull complete
Digest: sha256:adf10cca964f99224ac4948e782c260c4c87e71958df334e8ca882ff85062ea
Status: Downloaded newer image for mongo:7
cb125741a5f94f2c8b22e4f12b4f84277879b70b4958c0be881e3f97a4dd63ab1
```

Figure 9 — Conteneur MongoDB lancé sur EC2

```
[ec2-user@ip-172-31-17-99 ~]$ docker run -d --name mongo-crud-app --network appnet -p 3000:3000 -e MONGO_URI="mongodb://mongo:27017" registry.gitlab.com/cyber2026/mongodbusersapp/mongo-crud-app:latest
bc3c9991672008337aaed2d9651084bb5bf1c43143fad3231c8f8dc6520b3237
[ec2-user@ip-172-31-17-99 ~]$ docker stop mongo-crud-app
mongo-crud-app
*[[[A[ec2-user@ip-172-31-17-99 docker rm mongo-crud-app
mongo-crud-app
[ec2-user@ip-172-31-17-99 ~]$ docker run -d \
> --name mongo-crud-app \
> --network appnet \
> -p 3000:3000 \
> -e MONGO_URI="mongodb://mongo:27017" \
> registry.gitlab.com/cyber2026/mongodbusersapp/mongo-crud-app:latest
5e845d2cbbf725d5b0fe0bd6f5e25e2ccaceffee37cd59d986f7ea2b9c767eaf
[ec2-user@ip-172-31-17-99 ~]$
```

Figure 10 — Application mongo-crud-app lancée sur EC2

2. Module : DevSecOps

Vous devez rendre un dépôt GitHub regroupant tous les pipelines réalisés durant le semestre.

- Les pipelines CI/CD
- Les configurations GitLab CI/CD
- Les outils DevSecOps utilisés
- Les scripts d'automatisation
- Les scans de sécurité
- Les déploiements Docker/Kubernetes
- Un rapport global unique

```
@monsecops-project/
├── Pipel1 final/
├── Pipel1 final2/
├── Pipel1 final3/
├── Pipel1 final4/
├── Pipel1 final5/
├── app-jenkins/
├── docker/
├── kubernetes/
├── monsecops/
├── docker/
├── rapport_final1.pdf
├── gitlab-ci.yml
├── docker-compose.yml
└── README.md
```

Le rapport final doit inclure :

- Présentation du pipeline
- Architecture DevSecOps
- Explication des stages CI/CD
- Outils utilisés
- Analyse des résultats des scans
- Captures d'écran
- Conclusion

Figure 11 — Application accessible dans le navigateur (SSH port forward)

4. Pipeline 3 — SecNotes : SAST SonarQube

Le **Pipeline 3** est le plus complet en termes de sécurité. Il porte sur l'application **SecNotes** (Node.js + MongoDB, intentionnellement vulnérable) et implémente 7 stages : installation, tests, SAST SonarQube, build Docker, déploiement Kubernetes, scan DAST OWASP ZAP et provisionnement Terraform.

4.1 Description & Stages

Stage	Job	Outil	Statut	Description
install	install	npm	■ PASSED	Installation des dépendances Node.js
test	test	Jest	■ PASSED	Tests unitaires auth + notes (mongodb-memory-server)
sast	sonarqube_scan	SonarQube	■ PASSED	Analyse statique SAST du code source
build	build	Docker	■ PASSED	Build image Docker secnotes:\$SHA
deploy	deploy	kubectl	→ main only	Apply K8s manifests (namespace secnotes)
zap	zap_dast_scan	OWASP ZAP	→ main only	Scan dynamique DAST de l'application
terraform	terraform	Terraform	Manuel	terraform init + plan (tfplan)

4.2 Docker & Kubernetes (Pipeline 3)

SecNotes est containerisée avec un Dockerfile basé sur **node:20**. Des **manifests Kubernetes** complets sont fournis (namespace, configmap, secret, deployment, service) pour un déploiement dans le namespace *secnotes*.

Dockerfile SecNotes :

```
FROM node:20 WORKDIR /app COPY package*.json ./ RUN npm install COPY . . EXPOSE 3000 CMD ["npm","start"] # Note: exécution en root (V7 – vulnérabilité intentionnelle)
```

Manifest K8s	Rôle
namespace.yaml	Crée le namespace 'secnotes'
configmap.yaml	Variables de configuration (PORT, NODE_ENV)
secret.yaml	Secrets encodés Base64 (MONGO_URI, JWT_SECRET)
deployment.yaml	Déploiement SecNotes (1 réplique, image locale)
service.yaml	Exposition du service (NodePort)

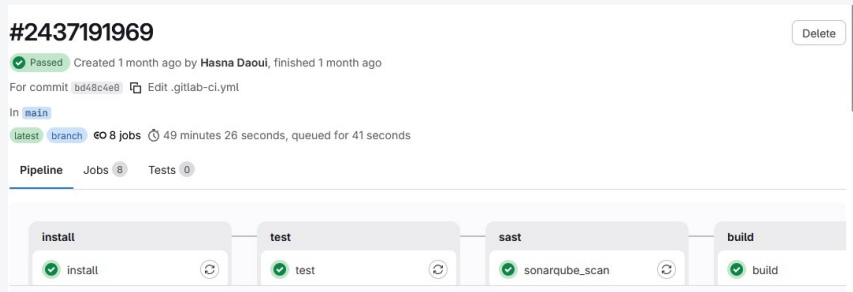
4.3 Résultats SAST — Vulnérabilités Détectées

L'analyse SonarQube a identifié **7 vulnérabilités de sécurité** dans le code source de SecNotes. Ces failles sont intentionnelles et couvrent les catégories OWASP Top 10 les plus critiques pour les applications web Node.js.

ID	Titre	Sévérité	Fichier Principal	OWASP Top 10
V1	Injection NoSQL	CRITIQUE	authController.js / noteController.js	A03:2021
V2	Secret JWT Hardcodé	CRITIQUE	middleware/auth.js	A02:2021
V3	Absence Validation Entrées	HAUTE	authController.js	A03/A04:2021
V4	IDOR — Contrôle d'Accès	HAUTE	noteController.js	A01:2021
V6	Hashage MD5 Faible	HAUTE	authController.js	A02:2021
V8	Logs Verbeux / Fuite Info	MOYENNE	app.js / controllers	A09:2021
V10	CORS Trop Permissif	MOYENNE	app.js	A05:2021

Sévérité	Nb	% du total
CRITIQUE (V1, V2)	2	28.6%
HAUTE (V3, V4, V6)	3	42.9%
MOYENNE (V8, V10)	2	28.6%
TOTAL	7	100%

4.4 Screenshots Pipeline 3



The screenshot displays a GitHub Actions workflow run for Pipeline 3, identified by ID #2437191969. The workflow is marked as 'Passed' and was created by Hasna Daoui. It is running on the 'main' branch for commit bd48c4e0. The workflow consists of four stages: 'install', 'test', 'sast', and 'build'. Each stage is represented by a box with a green checkmark, indicating successful completion. The 'install' stage contains the 'install' job, 'test' contains 'test', 'sast' contains 'sonarqube_scan', and 'build' contains 'build'. The workflow summary shows 8 jobs and 0 tests. The caption below the screenshot reads: 'Figure 12 — Pipeline 3 PASSED : 4 stages visibles (install, test, sast, build)'.

Figure 12 — Pipeline 3 PASSED : 4 stages visibles (install, test, sast, build)

5. Vue Globale : Docker Across All Pipelines

Les trois pipelines utilisent Docker comme outil de containerisation central. Voici une synthèse comparative des configurations Docker employées.

Critère	Pipeline 1 — Todo App	Pipeline 2 — MongoDB CRUD	Pipeline 3 — SecNotes
Image base	node:18-alpine	node:18-alpine	node:20 (non-alpine)
Port	8000	3000	3000
CMD	node server.js	npm start	npm start
Multi-stage	Non	Non	Non (V7 — vulnérable)
User non-root	Non	Non	Non (V7 — intentionnel)
Registry	GitLab Registry	GitLab Registry	Local (runner)
Docker Compose	Non	Oui (dev local)	Non
DinD (CI)	Oui	Oui	Non (runner local)
Déploiement	Staging (docker run)	K8s + EC2	K8s (namespace secnotes)

La progression entre les pipelines illustre l'évolution des bonnes pratiques : Pipeline 1 utilise une image Alpine minimale (sécurisée), Pipeline 2 ajoute Docker Compose et K8s, Pipeline 3 introduit intentionnellement des anti-patterns (root, node:20 non-alpine) pour être détectés par l'analyse de sécurité.

6. Scripts d'Automatisation

Deux scripts Bash d'automatisation ont été développés pour compléter les pipelines CI/CD.

deploy.sh — Script de Déploiement

Ce script automatise le déploiement de l'application Todo App dans deux environnements (staging ou production). Il gère le cycle de vie du conteneur Docker (stop, rm, run) et, en production, applique les manifests Kubernetes.

```
Usage : ./deploy.sh [staging|production] [version]
Fonctionnalités : - Staging : docker stop/rm + docker run -p 8001:8000 - Production : kubectl apply deployment + service + rollout status
```

security-scan.sh — Suite de Scans de Sécurité

Ce script orchestre 5 outils de sécurité en séquence et génère des rapports JSON horodatés dans un dossier security-reports/.

Étape	Outil	Type	Output
1/5	Gitleaks	Secrets scan	gitleaks-TIMESTAMP.json
2/5	npm audit	Dépendances CVE	npm-audit-TIMESTAMP.json
3/5	Trivy	Image Docker scan	trivy-TIMESTAMP.json
4/5	Hadolint	Lint Dockerfile	hadolint-TIMESTAMP.json
5/5	Python summary	Synthèse	summary-TIMESTAMP.json

7. Conclusion Générale

Ce module DevSecOps a permis de mettre en pratique l'ensemble des concepts fondamentaux de l'ingénierie de la sécurité moderne, en construisant progressivement trois pipelines CI/CD de complexité croissante sur GitLab.

Pipeline 1 a établi les bases d'un pipeline CI/CD sécurisé avec détection de secrets (Gitleaks), scan de dépendances (npm audit), containerisation Docker et déploiement automatisé en staging.

Pipeline 2 a étendu l'approche vers l'orchestration Kubernetes (Docker Desktop) et le provisionnement cloud AWS EC2 via Terraform, illustrant le cycle complet Build → Push Registry → Deploy K8s → Deploy EC2.

Pipeline 3 a intégré l'analyse SAST complète avec SonarQube, identifiant 7 vulnérabilités (2 critiques, 3 hautes, 2 moyennes) dans l'application SecNotes, et ajouté le scan DAST via OWASP ZAP.

Compétences acquises :

- Conception et implémentation de pipelines CI/CD multi-stages avec GitLab
- Containerisation Docker et gestion des registries d'images
- Orchestration Kubernetes avec manifests YAML (namespace, deployment, service, secret)
- Infrastructure-as-Code avec Terraform (AWS EC2, providers Kubernetes)
- Intégration d'outils de sécurité : Gitleaks, npm audit, SonarQube, OWASP ZAP
- Analyse et remédiation de vulnérabilités OWASP Top 10
- Approche Shift-Left Security : sécurité intégrée dès la phase CI/CD
- Scripting Bash pour l'automatisation DevSecOps